

Practical 3 — Descriptive Statistics (Measures of Central Tendency)

Q: What is the mean and when can it be misleading?

A: The mean is the arithmetic average of all values (sum divided by count). It can be misleading when data contains extreme outliers — for example, average income in a country is pulled up by a few billionaires, making the median a better representative measure.

Q: What is the difference between mean, median, and mode?

A: Mean is the arithmetic average. Median is the middle value when data is sorted — unaffected by outliers. Mode is the most frequently occurring value, useful for categorical data. For a symmetric normal distribution, all three are equal.

Q: What is standard deviation and what does it tell us?

A: Standard deviation (SD) measures how spread out values are from the mean. A small SD means values cluster tightly around the mean; a large SD means values are widely spread. Formula: $SD = \sqrt{\text{variance}} = \sqrt{(\text{sum of squared deviations} / n)}$.

Q: What is a percentile? How is the 75th percentile calculated?

A: A percentile indicates the value below which a given percentage of data falls. The 75th percentile (Q3) means 75% of the data is below this value. It is calculated by sorting data and finding the value at position $(75/100) \times n$.

Q: Why do we group by a categorical variable before computing summary statistics?

A: Grouping reveals patterns hidden in the overall summary. For example, average exam marks differ significantly between male and female students or different age groups. The grouped statistics expose these differences that an overall mean would mask.

Q: Explain the Iris dataset — how many classes, features, and samples?

A: The Iris dataset has 150 samples, 4 numeric features (sepal length, sepal width, petal length, petal width in cm), and 3 classes: Iris-setosa, Iris-versicolor, and Iris-virginica — 50 samples each. It is a classic benchmark for classification algorithms.

Q: What does the describe() function in pandas return?

A: describe() returns count, mean, std, min, 25th percentile (Q1), 50th percentile (median), 75th percentile (Q3), and max for each numeric column. It does NOT show mode, skewness, or missing value counts.

Practical 8 — Data Visualization I (Titanic – Histogram)

Q: What is a histogram and how does it differ from a bar chart?

A: A histogram displays the distribution of a continuous numeric variable by grouping data into bins (intervals) and showing frequency per bin. A bar chart shows counts or values for discrete categories. Histograms have no gaps between bars (continuous data); bar charts typically do.

Q: What does the bin width of a histogram affect?

A: Bin width controls the granularity of the distribution. Too wide: details are lost and the histogram looks flat. Too narrow: it looks noisy with many empty bins. The Freedman-Diaconis rule or Sturges' formula are used to choose optimal bin width.

Q: Why use Seaborn instead of Matplotlib for statistical plots?

A: Seaborn is built on Matplotlib and provides higher-level functions specifically designed for statistical visualization. It integrates natively with pandas DataFrames, automatically computes distributions, and produces aesthetically better plots with less code.

Q: What does a right-skewed distribution in the 'fare' column indicate?

A: Right (positive) skew means most passengers paid low fares, but a few paid very high fares — the tail extends to the right. This is typical for income and price data. The mean is greater than the median in a right-skewed distribution.

Q: Name at least 5 columns in the Titanic dataset.

A: survived (0/1), pclass (ticket class: 1/2/3), sex, age, sibsp (siblings/spouses aboard), parch (parents/children aboard), fare, embarked (port: C/Q/S), alone, deck.

Q: How do you handle outliers visible in a histogram?

A: Options include: (1) Remove them if they are data entry errors, (2) Cap them at a percentile threshold (Winsorization), (3) Apply log transformation to compress the range, or (4) Keep them if they are genuine extreme values that must be retained for analysis.

Practical 9 — Data Visualization II (Box Plot)

Q: What are the 5 components of a box plot?

A: The five components are: (1) Minimum — smallest non-outlier value, (2) Q1 — 25th percentile (lower hinge), (3) Median (Q2) — 50th percentile, the line inside the box, (4) Q3 — 75th percentile (upper hinge), (5) Maximum — largest non-outlier value. Outliers are plotted as individual points beyond the whiskers.

Q: What is the IQR and how is it used to detect outliers?

A: IQR (Interquartile Range) = $Q3 - Q1$. It measures the spread of the middle 50% of data. Outlier boundaries: Lower fence = $Q1 - 1.5 \times IQR$, Upper fence = $Q3 + 1.5 \times IQR$. Any point below the lower fence or above the upper fence is flagged as an outlier.

Q: How does a box plot show outliers?

A: Box plot whiskers extend to the most extreme non-outlier values (within $1.5 \times IQR$ from Q1 and Q3). Data points outside this range are plotted individually as dots or circles, making outliers immediately visible.

Q: What observations can you make about Titanic survival vs age/gender?

A: Female passengers had a much higher survival rate than males (reflecting the 'women and children first' policy). Children (lower age) had higher survival rates than adults. Older males in 3rd class had the lowest survival rate. The median age of survivors was slightly younger than non-survivors.

Q: What is the difference between a box plot and a violin plot?

A: A box plot shows only summary statistics (Q1, median, Q3, outliers). A violin plot shows the full probability density of the distribution (like a rotated KDE curve mirrored on both sides) in addition to the box plot summary. Violin plots reveal bimodality and distribution shape, which box plots cannot show.

Practical 10 — Data Visualization III (Iris Dataset)

Q: What are the 4 features in the Iris dataset and their types?

A: The four features are: sepal length (cm) – numeric/continuous, sepal width (cm) – numeric/continuous, petal length (cm) – numeric/continuous, petal width (cm) – numeric/continuous. The class label (species) is a nominal categorical variable.

Q: How do you identify a bimodal distribution from a histogram?

A: A bimodal distribution has two distinct peaks (modes) in the histogram. For example, petal length in the Iris dataset shows a bimodal distribution because Iris-setosa has much shorter petals (~1–2 cm) while versicolor and virginica cluster at higher values (~3–7 cm).

Q: Which Iris feature best separates the 3 species?

A: Petal length and petal width best separate the three species. Iris-setosa is clearly distinct (very short petals), while versicolor and virginica overlap slightly but are separated by petal size. Sepal width is the least discriminative feature.

Q: What is a pairplot and when would you use it?

A: A pairplot (seaborn.pairplot) creates a grid of scatter plots for every pair of features in a dataset, with histograms or KDE plots on the diagonal. It is used for exploratory data analysis to visualize relationships between all feature pairs simultaneously and identify which features best separate classes.

Q: What is the difference between nominal and numeric variables?

A: Nominal variables are categorical with no inherent order (e.g., species: setosa/versicolor/virginica, color: red/blue). Numeric variables have quantitative values — either continuous (e.g., petal length with infinite precision) or discrete (e.g., count of petals as integers).

MEDIUM — Practicals 1, 2, 7 & Big Data B1–B3

Practical 1 — Data Wrangling I

Q: What is data wrangling and why is it needed?

A: Data wrangling (also called data munging) is the process of cleaning, transforming, and structuring raw data into a suitable format for analysis. Real-world data is often messy — it has missing values, wrong data types, inconsistent formats, duplicates, and outliers. Without wrangling, ML models produce unreliable results (garbage in, garbage out).

Q: What is the difference between `isnull()` and `isna()` in pandas?

A: They are identical — `isna()` is an alias for `isnull()`. Both return a Boolean DataFrame/Series where True indicates a missing value (NaN or None). You can use `df.isnull().sum()` to count missing values per column.

Q: What is data normalization? Compare min-max and Z-score normalization.

A: Normalization scales numeric features to a standard range. Min-max normalization scales to $[0, 1]$: $X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$. Z-score standardization transforms to mean=0, std=1: $X_{\text{std}} = (X - \text{mean}) / \text{std}$. Use min-max when you need values in a bounded range; use Z-score when the algorithm assumes normally distributed data (e.g., SVM, linear regression).

Q: What is label encoding vs one-hot encoding?

A: Label encoding assigns a unique integer to each category (e.g., cat=0, dog=1, bird=2). It is suitable for ordinal variables but can mislead models into thinking there is an order. One-hot encoding creates a binary column for each category (e.g., `is_cat`, `is_dog`, `is_bird`). It is used for nominal variables to avoid false ordinal relationships.

Q: What does the `describe()` output and what does it NOT show?

A: `describe()` shows count, mean, std, min, Q1 (25%), median (50%), Q3 (75%), and max for numeric columns. It does NOT show: mode, skewness, kurtosis, missing value count (you need `isnull().sum()` for that), or non-numeric column statistics (unless you add `include='all'`).

Q: What are the types of data variables?

A: Types include: Nominal (categories with no order — gender, city), Ordinal (ordered categories — low/medium/high, education level), Interval (numeric with no true zero — temperature in Celsius), Ratio (numeric with true zero — height, weight, income). In pandas, types are: `int64`, `float64` (numeric), `object` (string/categorical), `bool`, `datetime64`.

Practical 2 — Data Wrangling II (Academic Performance)

Q: What is an outlier and what techniques are used to detect them?

A: An outlier is a data point that deviates significantly from other observations. Detection methods: (1) Z-score method — values with $|z| > 3$ are outliers, (2) IQR method — values outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$, (3) Box plot — visual detection, (4) Scatter plot — visual for bivariate outliers. IQR method is preferred for skewed data; Z-score assumes normality.

Q: How do you handle outliers?

A: Options: (1) Remove — only if confirmed as data entry error, (2) Winsorization/Capping — replace values beyond a threshold with the threshold value (e.g., cap at 95th percentile), (3) Log transformation — compresses extreme values, (4) Treat separately — model with and without outliers, (5) Imputation — replace with median. Never blindly remove outliers without justification.

Q: What is data transformation? Give examples.

A: Data transformation changes the scale or distribution of a variable. Examples: Log transformation ($\log(X)$) — reduces right skew, makes multiplicative relationships additive; Square root ($\sqrt{X}$) — moderate skew reduction; Square (X^2) — amplifies differences for right-skewed left data; Box-Cox transformation — general parametric method. Purpose: achieve normality, improve model performance, make patterns linear.

Q: What is skewness? Explain positive vs negative skew.

A: Skewness measures the asymmetry of a distribution. Positive (right) skew: long tail on the right, $\text{mean} > \text{median} > \text{mode}$ — common in income, age, fare data. Negative (left) skew: long tail on the left, $\text{mean} < \text{median} < \text{mode}$ — e.g., exam marks when most students score high. Skewness = 0 for a symmetric (normal) distribution. Apply log transform for positive skew.

Q: When is log transformation useful and why?

A: Log transformation is useful for: (1) Right-skewed data (income, population, prices) to make distribution more normal, (2) Converting multiplicative relationships to additive ($Y = a \times X^b \rightarrow \log Y = \log a + b \times \log X$), (3) Reducing the influence of extreme outliers. Constraint: log is only defined for positive values (use $\log(X+1)$ if data contains zeros).

Q: Techniques for handling missing values.

A: Options: (1) Deletion — drop rows (listwise) or columns with many missing values, (2) Mean/Median imputation — replace with mean (for symmetric data) or median (for skewed data), (3) Mode imputation — for categorical variables, (4) Forward/Backward fill — for time-series data, (5) KNN imputation — use K nearest neighbors, (6) Prediction model — predict missing values using other features.

Q: What is the difference between data cleaning and data transformation?

A: Data cleaning fixes errors and inconsistencies: removing duplicates, correcting typos, handling missing values, fixing wrong data types. Data transformation changes the scale or structure for analysis purposes: normalization, log transform, encoding categoricals, creating new features. Cleaning ensures data quality; transformation prepares data for modeling.

Practical 7 — Text Analytics (NLP)

Q: What is tokenization?

A: Tokenization is the process of splitting text into smaller units called tokens. Word tokenization splits text into individual words (e.g., 'I love NLP' → ['I', 'love', 'NLP']). Sentence tokenization splits text into sentences. It is the first step in NLP preprocessing. NLTK provides `word_tokenize()` and `sent_tokenize()`.

Q: What are stop words and why are they removed?

A: Stop words are common words that appear frequently but carry little semantic meaning (e.g., 'is', 'the', 'a', 'an', 'in', 'on'). They are removed to reduce noise and dimensionality so the model focuses on meaningful words. NLTK provides a predefined stop word list for many languages via `stopwords.words('english')`.

Q: What is the difference between stemming and lemmatization?

A: Stemming chops off word endings using rules to get the root form — fast but can produce non-words (e.g., 'running' → 'run', 'studies' → 'studi', 'caring' → 'car'). Lemmatization uses vocabulary and grammar to return the actual dictionary base form (e.g., 'running' → 'run', 'studies' → 'study', 'caring' → 'care'). Lemmatization is more accurate but slower. NLTK provides `PorterStemmer` and `WordNetLemmatizer`.

Q: What is POS tagging? Give common tags.

A: POS (Part-of-Speech) tagging assigns grammatical tags to each word. Common tags: NN (noun singular), NNS (noun plural), VB (verb base), VBG (verb gerund), JJ (adjective), RB (adverb), DT (determiner), IN (preposition), PRP (personal pronoun). NLTK's `pos_tag()` function performs POS tagging. It is used for named entity recognition, parsing, and text understanding.

Q: What is TF-IDF? Write the formula.

A: TF-IDF (Term Frequency–Inverse Document Frequency) is a numerical statistic that reflects how important a word is to a document relative to a corpus. $TF(t,d) = (\text{occurrences of term } t \text{ in doc } d) / (\text{total terms in } d)$. $IDF(t) = \log(N / df(t))$ where N = total documents, $df(t)$ = documents containing t . $TF-IDF = TF \times IDF$. High TF-IDF means the word is frequent in that document but rare overall — indicating high relevance.

Q: Why is TF alone not enough — what does IDF add?

A: TF alone would give high scores to common words like 'the', 'is', 'a' that appear in almost every document. IDF penalizes words that appear across many documents (high df) and rewards rare words that appear in few documents. This ensures common words get low TF-IDF scores while distinctive topic words get high scores.

Q: What is a bag-of-words model?

A: Bag-of-words (BoW) represents a document as a vector of word counts, ignoring grammar and word order. For example, 'the cat sat on the mat' → {the: 2, cat: 1, sat: 1, on: 1, mat: 1}. It is simple but loses semantic relationships. TF-IDF improves BoW by weighting terms. `Word2Vec` and `BERT` improve further by capturing context and meaning.

Big Data — Practicals B1, B2, B3 (Hadoop MapReduce)

Q: What is Hadoop and what are its main components?

A: Apache Hadoop is an open-source framework for distributed storage and processing of large datasets across clusters. Main components: (1) HDFS (Hadoop Distributed File System) — distributed storage with replication, (2) MapReduce — distributed computation model, (3) YARN (Yet Another Resource Negotiator) — cluster resource management, (4) Common — shared utilities and libraries.

Q: Explain the Map and Reduce phases with an example (WordCount).

A: In WordCount: Map phase — reads input splits, tokenizes each line, emits (word, 1) key-value pairs. Shuffle and Sort phase — groups all values for the same key together. Reduce phase — for each word key, sums all the 1s and emits (word, total_count). Example: input 'hello world hello' → Map emits (hello,1),(world,1),(hello,1) → Shuffle groups → Reduce outputs (hello,2),(world,1).

Q: What is the Shuffle and Sort phase in MapReduce?

A: After the Map phase, the framework automatically sorts and groups all key-value pairs by key across all mapper outputs. All values with the same key are grouped together and sent to the same Reducer. This phase is implicit — the developer does not write code for it. It ensures each Reducer receives a sorted list of (key, [list of values]).

Q: What is a Combiner in MapReduce?

A: A Combiner is a local mini-reducer that runs on the Mapper node before data is sent across the network. It performs partial aggregation locally to reduce network traffic. For WordCount, a Combiner sums word counts on the local node before sending to the Reducer. Combiners work only for commutative and associative operations (sum, count, max, min — but not average).

Q: What is YARN and what is the role of the NameNode?

A: YARN (Yet Another Resource Negotiator) manages cluster resources. It has: ResourceManager (global cluster resource allocator) and NodeManager (per-node resource management). The NameNode is the master daemon in HDFS — it stores metadata (file names, block locations, replication factor) but NOT actual data. DataNodes store actual data blocks. Loss of NameNode is catastrophic, so it has a Secondary NameNode or HA standby.

Q: What is the difference between standalone, pseudo-distributed, and fully-distributed Hadoop?

A: Standalone mode: single Java process, no HDFS — for testing and debugging. Pseudo-distributed: all Hadoop daemons run on a single machine in separate JVMs — simulates a cluster environment. Fully-distributed: daemons run on multiple machines — production setup with true parallel processing and fault tolerance.

Q: What is HDFS and how does it handle fault tolerance?

A: HDFS stores large files by splitting them into blocks (default 128 MB) and distributing them across DataNodes. Fault tolerance: each block is replicated across 3 DataNodes by default (replication factor = 3). If a DataNode fails, the NameNode detects it via heartbeat and re-replicates the lost blocks from surviving replicas automatically.

Practical 4 — Data Analytics I (Linear Regression — Boston Housing)

Q: What is linear regression? Difference between simple and multiple.

A: Linear regression models the linear relationship between a dependent variable (Y) and one or more independent variables (X). Simple linear regression: $Y = \beta_0 + \beta_1 X + \varepsilon$ (one predictor). Multiple linear regression: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$ (multiple predictors). The goal is to find coefficients β that minimize the residual sum of squares.

Q: What is the cost function and how does gradient descent minimize it?

A: The cost function for linear regression is MSE (Mean Squared Error): $MSE = (1/n) \times \sum (Y_i - \hat{Y}_i)^2$. Gradient descent minimizes MSE by iteratively updating coefficients: $\beta = \beta - \alpha \times (\partial MSE / \partial \beta)$, where α is the learning rate. It moves in the direction of steepest descent until convergence. Alternative: Normal Equation $\beta = (X^T X)^{-1} X^T Y$ solves directly without iteration.

Q: What are the assumptions of linear regression?

A: Five key assumptions: (1) Linearity — relationship between X and Y is linear, (2) Independence — observations are independent of each other, (3) Homoscedasticity — constant variance of residuals across all X values, (4) Normality — residuals are normally distributed, (5) No multicollinearity — predictors are not highly correlated with each other.

Q: What is R² score and what does R²=0.85 mean?

A: R² (coefficient of determination) measures the proportion of variance in Y explained by the model. $R^2 = 1 - (SS_{res} / SS_{tot})$. R²=0.85 means the model explains 85% of the variance in the target variable — the remaining 15% is unexplained. R²=1 is a perfect fit; R²=0 means the model does no better than the mean. Adjusted R² penalizes for adding irrelevant features.

Q: What is overfitting and underfitting?

A: Overfitting: the model learns training data too well including noise — high accuracy on training set but poor on test set. Caused by too many features or a model too complex. Underfitting: the model is too simple to capture patterns — poor accuracy on both training and test sets. Detected using learning curves. Solutions: regularization (Ridge/Lasso), cross-validation, pruning, more training data.

Q: What does each important Boston Housing feature represent?

A: CRIM — per capita crime rate by town; RM — average number of rooms per dwelling (most positively correlated with house price); LSTAT — % of population with lower status (most negatively correlated with price); NOX — nitric oxide concentration (air pollution); DIS — weighted distance to employment centers; TAX — property tax rate per \$10,000; MEDV — median home value in \$1000s (target variable).

Q: What is multicollinearity and how does it affect regression?

A: Multicollinearity occurs when two or more independent variables are highly correlated with each other. This makes it difficult to determine each variable's individual effect on Y — coefficients become unstable and have high variance. Detection: VIF (Variance Inflation Factor) — VIF > 10 indicates serious multicollinearity. Solutions: remove one of the correlated variables, use Ridge regression (L2 regularization), or PCA.

Practical 5 — Data Analytics II (Logistic Regression — Social Network Ads)

Q: Why can't we use linear regression for classification?

A: Linear regression outputs a continuous value (e.g., $-\infty$ to $+\infty$), which cannot represent probabilities. For binary classification, we need output in $[0, 1]$. Also, linear regression is sensitive to outliers, violates the assumption of binary targets, and the decision boundary is not optimal. Logistic regression uses the sigmoid function to squash output to $(0, 1)$ and models class probability.

Q: What is the sigmoid function and what does it output?

A: The sigmoid (logistic) function is $\sigma(z) = 1 / (1 + e^{-z})$, where $z = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$. It outputs values strictly between 0 and 1, interpreted as the probability $P(Y=1|X)$. If $\sigma(z) \geq 0.5$ (i.e., $z \geq 0$), predict class 1; otherwise predict class 0. The decision boundary is a straight line/hyperplane where $z = 0$.

Q: Explain the confusion matrix and define TP, FP, TN, FN.

A: The confusion matrix is a 2×2 table comparing predicted vs actual classes. TP (True Positive): model predicted Positive and it is actually Positive. TN (True Negative): predicted Negative, actually Negative. FP (False Positive / Type I Error): predicted Positive, actually Negative. FN (False Negative / Type II Error): predicted Negative, actually Positive. The diagonal (TP, TN) shows correct predictions; off-diagonal (FP, FN) shows errors.

Q: Write formulas for Accuracy, Precision, Recall, and F1-score.

A: Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ — overall correctness. Precision = $TP / (TP + FP)$ — of all positive predictions, how many are correct. Recall (Sensitivity) = $TP / (TP + FN)$ — of all actual positives, how many did we catch. F1-score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — harmonic mean of precision and recall, useful when classes are imbalanced.

Q: When do you prioritize Precision vs Recall?

A: Prioritize Precision when false positives are costly — e.g., spam detection (marking a legitimate email as spam is bad), medical drug approval (approving a harmful drug). Prioritize Recall when false negatives are costly — e.g., cancer screening (missing a cancer patient is dangerous), fraud detection (missing a fraud transaction). F1-score balances both when neither error is worse.

Q: What is an ROC curve and what does AUC tell you?

A: The ROC (Receiver Operating Characteristic) curve plots True Positive Rate (Recall) vs False Positive Rate at various classification thresholds. AUC (Area Under Curve) measures overall model performance regardless of threshold. AUC = 0.5 means random guessing; AUC = 1.0 means perfect classifier. AUC > 0.9 is excellent; AUC > 0.8 is good. Useful for comparing models on imbalanced datasets.

Q: What is the Social Network Ads dataset?

A: The Social Network Ads dataset contains information about users from a social network and whether they purchased a product based on an advertisement. Features: User ID, Gender, Age, Estimated Salary. Target: Purchased (0 or 1). It is used to demonstrate binary classification — predicting if a user will buy a product based on demographic information.

Practical 6 — Data Analytics III (Naïve Bayes — Iris)

Q: What is Bayes' theorem? State the formula.

A: Bayes' theorem describes the probability of an event given prior knowledge. Formula: $P(A|B) = [P(B|A) \times P(A)] / P(B)$. In classification: $P(\text{Class}|\text{Features}) = [P(\text{Features}|\text{Class}) \times P(\text{Class})] / P(\text{Features})$. $P(\text{Class})$ = prior probability (class frequency), $P(\text{Features}|\text{Class})$ = likelihood, $P(\text{Features})$ = evidence (constant for all classes), $P(\text{Class}|\text{Features})$ = posterior probability. We predict the class with the highest posterior.

Q: What is the 'Naïve' assumption in Naïve Bayes?

A: The 'Naïve' assumption is that all features are conditionally independent given the class label. This means: $P(X_1, X_2, \dots, X_n | C) = P(X_1 | C) \times P(X_2 | C) \times \dots \times P(X_n | C)$. This simplification makes computation tractable but is rarely true in practice (features are often correlated). Despite this unrealistic assumption, Naïve Bayes works surprisingly well in text classification and other high-dimensional tasks.

Q: Types of Naïve Bayes — when to use each?

A: Gaussian Naïve Bayes: features are continuous and normally distributed — used for numeric data like the Iris dataset. Multinomial Naïve Bayes: features are counts (term frequencies) — used for text classification, document categorization. Bernoulli Naïve Bayes: features are binary (0/1) — used for binary feature text data (word presence/absence, spam detection). For the Iris dataset, Gaussian NB is used because all 4 features are continuous measurements.

Q: What is prior probability vs posterior probability?

A: Prior probability $P(C)$ is the initial probability of a class based only on class frequency in training data, without considering any features. For example, if 33% of Iris samples are setosa, $P(\text{setosa}) = 0.33$. Posterior probability $P(C|X)$ is the updated probability after observing the features. Naïve Bayes classifies by finding the class with the maximum posterior: $\text{argmax } P(C|X) = \text{argmax } [P(X|C) \times P(C)]$.

Q: What are the advantages and disadvantages of Naïve Bayes?

A: Advantages: (1) Very fast to train and predict, (2) Works well with small training data, (3) Handles high-dimensional data (text), (4) Naturally handles multi-class problems, (5) Not sensitive to irrelevant features. Disadvantages: (1) The independence assumption is almost never true, (2) Cannot learn feature interactions, (3) For numeric features, assumes a specific distribution (Gaussian), (4) Zero-frequency problem for unseen category values.

Q: What is the zero-frequency problem and how is Laplace smoothing used?

A: Zero-frequency problem: if a feature value was never observed with a particular class in training data, $P(\text{feature value} | \text{class}) = 0$, making the entire posterior probability zero regardless of other features. Laplace smoothing (Add-1 smoothing): adds 1 to every count: $P(X_i|C) = (\text{count}(X_i, C) + 1) / (\text{count}(C) + \text{vocabulary size})$. This ensures no probability is exactly zero.

Q: Compare Naïve Bayes vs Logistic Regression.

A: Naïve Bayes is a generative model (models $P(X|Y)$ and $P(Y)$) — fast, works with tiny datasets, probabilistic output, handles missing features. Logistic Regression is a discriminative model (models $P(Y|X)$ directly) — generally more accurate with sufficient data, does not assume feature independence, better calibrated probabilities, can learn feature interactions via polynomial features. For text classification: NB often wins with small data; LR wins with large data.

Big Data — Practical B4 (Apache Spark in Scala)

Q: What is Apache Spark and how is it different from Hadoop MapReduce?

A: Apache Spark is an open-source, in-memory distributed computing framework. Key differences: Spark performs computation in-memory (RAM) while Hadoop MapReduce writes intermediate results to disk — making Spark 10–100× faster. Spark supports iterative algorithms (ML, graph processing) natively. Spark has a rich API (DataFrames, SQL, MLlib, Streaming) vs MapReduce's rigid Map-Reduce paradigm. Spark can also run on top of HDFS, YARN, Mesos, or Kubernetes.

Q: What is an RDD (Resilient Distributed Dataset)?

A: RDD is the fundamental data structure of Spark — an immutable, distributed collection of objects partitioned across cluster nodes. Resilient: fault-tolerant through lineage (DAG of transformations). Distributed: data is split into partitions across nodes. Dataset: typed collection of any Java/Scala/Python objects. RDDs support two types of operations: Transformations (lazy, return new RDD) and Actions (trigger computation, return values).

Q: Difference between transformation and action in Spark.

A: Transformations are lazy operations that define a new RDD from an existing one but do not execute immediately: `map()`, `filter()`, `flatMap()`, `groupByKey()`, `reduceByKey()`. Actions trigger actual computation and return results to the driver or write to storage: `collect()`, `count()`, `reduce()`, `saveAsTextFile()`, `take(n)`, `first()`. Spark builds a DAG of transformations and only executes when an action is called — this is lazy evaluation.

Q: What is lazy evaluation in Spark?

A: Lazy evaluation means Spark does not execute transformations immediately when they are defined. Instead, it records the operations in a DAG (Directed Acyclic Graph). Execution only happens when an action (e.g., `collect()`, `count()`) is called. Benefits: (1) Spark can optimize the full execution plan before running, (2) Unnecessary computations can be skipped, (3) Operations can be pipelined to avoid creating intermediate RDDs.

Q: What is a SparkContext vs SparkSession?

A: SparkContext (SC) was the original entry point to Spark functionality in Spark 1.x — it represented the connection to the cluster and was used to create RDDs. SparkSession was introduced in Spark 2.0 as the unified entry point that subsumes SparkContext, SQLContext, and HiveContext. SparkSession provides access to DataFrames, SQL, streaming, and all Spark features. It internally creates a SparkContext.

Q: What is a DAG in Spark? How does it help with fault tolerance?

A: DAG (Directed Acyclic Graph) is a logical execution plan that records all the transformations applied on RDDs. Each node is an RDD and each edge is a transformation. When a partition is lost due to node failure, Spark recomputes it by replaying the transformations from the DAG (lineage) rather than requiring data replication like HDFS. This makes Spark fault-tolerant without storing redundant copies in memory.

Q: Why is Scala preferred for Spark?

A: Spark is written in Scala, so Scala has the most complete and up-to-date API. Scala is statically typed, compiled to JVM bytecode — fast performance close to Java. Scala supports functional programming (`map`, `filter`, `reduce`, lambdas) natively, which aligns perfectly with Spark's functional API. Scala's REPL (`spark-shell`) provides interactive development. Python

(PySpark) is also popular but has some performance overhead due to serialization between JVM and Python.